

Desenvolvimento de Quiz em C: Uma Abordagem Inovadora

Bruno Salvador

Paulo Lucas

Professor: Aldo Henrique

Centro universitário Unieuro

bruno090507@unieuro.com.br

paulo093166@unieuro.com.br

I. INTRODUÇÃO

A linguagem C é amplamente reconhecida por seu alto desempenho e pelo controle granular que oferece ao programador sobre os recursos do sistema [1]. Em disciplinas de Estrutura de Dados, compreender como agrupar informações heterogêneas e manipular seus endereços de memória é fundamental para o desenvolvimento de softwares eficientes e otimizados [2].

Neste contexto, o presente projeto tem como objetivo desenvolver um jogo interativo do tipo "Quiz", focado em testar conhecimentos sobre a própria linguagem C. O programa visa não apenas criar uma interface funcional no terminal, mas aplicar na prática conceitos fundamentais da programação estruturada, rompendo com o uso de escopos estáticos e variáveis globais soltas.

Para alcançar este objetivo, o sistema foi projetado para abrigar um banco de 30 questões. A cada rodada, o programa embaralha este banco e seleciona apenas 10 perguntas, garantindo uma experiência dinâmica ao usuário. O desenvolvimento exigiu a implementação de alocação dinâmica de memória, passagem de parâmetros por referência e tratamentos rigorosos no fluxo de entrada de dados do teclado.

II. METODOLOGIA

A arquitetura do código foi construída sob quatro pilares principais:

Estruturação de Dados (Structs): Os dados foram encapsulados via typedef struct em uma estrutura denominada Pergunta, que agrupa um vetor para o enunciado, uma matriz para três alternativas de resposta e um char para o gabarito. Isso facilitou enormemente a manipulação do banco de 30 questões originais.

Alocação Dinâmica (Malloc e Free): O programa não instancia todas as questões simultaneamente na memória para o jogo. Ele aloca espaço estritamente para as 10 perguntas da rodada utilizando malloc e o operador sizeof(Pergunta). Ao final do laço de execução, a função free é acionada para

devolver a memória ao Sistema Operacional, prevenindo o vazamento de memória (*memory leak*).

Ponteiros e Passagem por Referência: A função modular fazerPergunta exibe e corrige as questões. Para evitar o alto custo computacional de copiar matrizes inteiras para a função, utilizou-se a passagem por referência recebendo o ponteiro Pergunta *p. Os membros são acessados pelo operador seta (->), e a chamada da função na main envia o endereço de memória usando o operador &.

Entrada de Dados e Limpeza de Buffer: A interação utiliza scanf para ler um caractere, deduzindo 32 do valor da Tabela ASCII inserido pelo usuário para validar tanto respostas minúsculas quanto maiúsculas de forma nativa. Além disso, a função de sanitização limparBuffer() varre e descarta caracteres residuais ou quebras de linha (\n) do teclado utilizando um laço while com a instrução getch(), prevenindo que o fluxo do programa salte leituras.

III. RESULTADOS

```
QUIZ DE C

1. Qual simbolo e usado para comentarios de uma linha?
a) //
b) /*
c) <!--
Sua resposta: 
```

Figura 1: Tela inicial do programa e exibição da primeira pergunta embaralhada.

```
QUIZ DE C

1. Qual simbolo e usado para comentarios de uma linha?
a) //
b) /*
c) <!--
Sua resposta: a
-> Acertou!

2. O que o comando 'return 0' indica na main?
a) Erro de compilacao
b) Execucao com sucesso
c) Loop infinito
Sua resposta: a
-> Errou! Era a letra 'b'.
```

Figura 2: Entrada de dados do usuário e feedback de acerto/erro (validação de caracteres).

```
FIM DE JOGO! Voce acertou 8 de 10.

Quer tentar de novo? (s/n): 
```

Figura 3: Saída final do programa, exibição da pontuação e menu de repetição (com limpeza de tela em ação).

IV. DISCUSSÃO

A implementação do quiz evidenciou que o controle manual de memória oferece eficiência superior a métodos puramente estáticos. O uso de structs garantiu a escalabilidade do

banco de questões, alinhando-se aos princípios organizacionais destacados por Schildt. O avanço central do projeto foi a alocação dinâmica com `malloc`, que otimizou o sistema ao reservar espaço estritamente para as 10 perguntas da rodada. Aliado a isso, a passagem por referência via ponteiros evitou cópias desnecessárias de dados e reduziu o consumo da *stack*, prática essencial defendida por Kernighan e Ritchie. A solução também se mostrou eficaz ao embaralhar o banco para evitar repetições e ao implementar a função `limparBuffer()`, mitigando as falhas clássicas de leitura do `scanf`. Embora o controle manual exija rigor para evitar vazamentos de memória (garantido pelo comando `free`), a aplicação conjunta dessas técnicas resultou em um software estável, robusto e profissional.

V. CONCLUSÃO

A elaboração deste projeto demonstrou, na prática, a importância do gerenciamento consciente da memória e do fluxo de dados em C. Compreendeu-se que o uso de structs torna o código legível e escalável, enquanto os ponteiros e a alocação dinâmica (`malloc/free`) são indispensáveis para criar aplicações eficientes que não sobrecarregam o hardware. Além disso, a implementação de tratamentos como a limpeza de buffer de teclado reforçou a necessidade de projetar softwares tolerantes a falhas na interação humana, resultando em uma aplicação robusta e funcional.

REFERÊNCIAS

- [1] KERNIGHAN, B. W.; RITCHIE, D. M. C: a linguagem de programação. 2. ed. Rio de Janeiro: Campus, 1989.
- [2] SCHILDT, H. C completo e total. 3. ed. São Paulo: Makron Books, 1997.
- [3] Material didático e de laboratório da disciplina de Algoritmos e Estrutura de Dados, Centro Universitário Unieuro, 2026.